

Testing with Spring

Unit, context, integration and web testing in the Spring ecosystem.

Strategy

Combine fast unit tests with context/integration tests only where they add confidence. Avoid starting the whole application when an isolated unit is enough.

Official site and documentation

Use the official Testing with Spring source as the primary reference:

<https://docs.spring.io/spring-framework/reference/testing.html>. Official documentation should be the first source for installation, supported versions, syntax changes and new features.

JUnit and Mockito

JUnit provides execution/assertions; Mockito supports isolation. Spring adds context and testing infrastructure.

Web and APIs

For web applications, validate controllers and endpoints at the appropriate level and complement with external API tests when needed.

Data and integrations

Control data, state, transactions and dependencies. Choose consciously between mocks, stubs, containers or real environments.

QA and quality

QA can contribute to strategy, identify integration risk and keep the pyramid from depending only on slow E2E tests.

Installation and configuration - overview

Always use the official website and documentation. Check prerequisites and compatible versions, install for your operating system, configure environment variables or PATH only when required, validate the installation with the recommended command or screen, and run a first project/test before integrating it into the QA workflow.

What it is and where it fits

Testing with Spring is a tool or technology that can support Quality Engineering work. Before adopting it, understand the problem it solves, where it fits in the workflow and what evidence or productivity gain it should provide.

How to start safely

Start from the official Testing with Spring site and documentation. Confirm prerequisites, supported versions, operating system and required permissions. Learn in a controlled environment before using it in a critical workflow.

First QA-oriented use

Pick one small, verifiable task in Testing with Spring, configure only what is necessary, run a simple example and confirm the outcome. Repeat with different data or conditions before adding abstractions.

Using it in a real project

Define where Testing with Spring fits, who owns configuration, how credentials and data are handled and how results are shared. Integrate it into the team workflow instead of creating an isolated activity.

Adoption good practices

Standardize versions and configuration, document important decisions, keep secrets out of code and preserve reproducible examples. Reassess whether Testing with Spring still delivers value as the product evolves.

Common issues and troubleshooting

When Testing with Spring fails, verify versions, dependencies, permissions, connectivity, data and environment before assuming a product defect. Preserve the original error and reduce the issue to the smallest reproducible case.